Flux de remboursement, correction et note de crédit

Date : 2026-10-03 **Code :** `crates/pos-engine/src/commands/refund.rs` **Périmètre cert :** M111 (remboursement complet), M112B (remboursement partiel - ticket séparé). La correction de paiement (M132, UC4118) n'est pas dans le périmètre de la v1.0.0 (voir §2). La note de crédit (UC4122) est dans le périmètre et rejouée (voir §3).

---

1. Implémentation du remboursement

La commande Tauri `refund_checkout` prend :

- `original_fiscal_event_id` - ligne locale de `fiscal_events` identifiant l'événement N à rembourser.
- `items: Vec<RefundItem>` - quoi rembourser (le ticket complet pour un remboursement total, un sous-ensemble pour un remboursement partiel).
- `employee_insz` - INSZ de l'opérateur, qui doit être le Gérant connecté (`PROFILS_UTILISATEURS_FR.md` §3).

Le flux reflète l'encaissement standard de `crates/pos-engine/src/commands/checkout.rs` :

1. **Chargement de l'original.** Résolution de `fdm_id`, `fdmDateTime`, `eventLabel`, `eventCounter`, `total-Counter` depuis la ligne `fiscal_events` locale. Seuls les événements N sont remboursables ; tout autre label retourne l'erreur codée `not_refundable|label=X`. Sont aussi refusés, avant toute création :
   - `refund_unpaid_sale` : une vente soldée sans encaissement (« impayée ») n'a pas pris d'argent, il n'y en a pas à rendre ;
   - `refund_of_refund` : un remboursement ne se rembourse pas (on revend) ;
   - `refund_in_training` : tout remboursement en mode formation (un événement T ne pourrait pas consommer la quantité encore remboursable d'une vente réelle) ;
   - `refund_discounted_line` : une ligne de commande en ligne portant une remise (la remise est répartie sur ses lignes : une partie rembourserait une part que personne ne peut établir) ; la commande entière se rembourse, remises comprises.
2. **Contrôles, puis construction de la commande de remboursement.** Chaque article est d'abord contrôlé (présent sur le ticket original, quantité positive, total remboursé sur l'ensemble des remboursements de la vente au plus égal à la quantité vendue), et le support du ticket choisi : un refus ne crée rien. Le compte se fait par seau (`Bucket` dans `refund.rs` : produit, prix unitaire et label de TVA) : l'écran envoie la ligne d'origine choisie (`lineId`), et l'article est remboursé au prix de cette ligne, chaque ligne d'origine étant reprise avec son propre prix. Un produit vendu à plusieurs prix sans ligne désignée est refusé (`refund_line_ambiguous`). Ensuite seulement, création d'une nouvelle `Order` (takeaway) avec des lignes de quantités négatives pour chaque `RefundItem`.
3. **Réservation du prochain numéro de ticket** (peek-then-commit, cf. `STRATEGIE_COMPTEURS_FR.md` §5). Label N.
4. **Construction du `SaleInput`** avec `build_sale_input` (9 paramètres), en passant la `FdmReference` originale comme dernier paramètre (`Some(&original_ref)`). Un remboursement en espèces est arrondi aux 5 centimes comme une vente en espèces : ligne `PAYMENT` du montant exact, puis ligne `ROUNDING` (exemple de ticket de remboursement du SPF : -3,04 rendus -3,05, arrondi -0,01). Le tiroir enregistre le montant rendu, arrondi compris. Le FDM utilise cette référence pour relier le remboursement à sa source - spec POS v2.2 §313 "fdmRef".
5. **Envoi au FDM.** Même chemin `signing::sign` qu'un encaissement normal. Le blocage sur déconnexion s'applique à l'identique : sans FDM, le remboursement est refusé (« FDM not connected - cannot process refund ») et la commande de remboursement est annulée (`voided`). Le remboursement saisi à la caisse se fait en espèces uniquement (`RefundSpec::cash`) ; seuls les remboursements d'une commande en ligne annulée peuvent repartir par la plateforme (`APP` ou `ONLINE`, voir `COMMANDES_EN_LIGNE_FR.md`).
6. **Archive + clôture atomiques.** Même pattern transaction SQLite unique que checkout (`signing::commit`) : `FiscalCounter::next` + `archive_event` + clôture de la commande + ligne `payments` dans une tx, rollback si l'un échoue. Cf. `STRATEGIE_COMPTEURS_FR.md` §5.

7. **Impression ticket de remboursement + ouverture tiroir.** Best-effort - non-fatal si imprimante injoignable. Chaque ligne remboursée est imprimée avec sa quantité et son montant négatifs (« -1 Cola -3,04 »), comme la requête M111 les transmet au FDM : le ticket reproduit les données signées.

Si le remboursement s'arrête après la création de sa commande (refus ou indisponibilité du FDM, échec de l'archivage), la commande est annulée (`status = 'voided'`) : elle ne reste jamais ouverte, et ne peut donc pas être complétée puis encaissée comme une vente.

Réponse perdue. Si la requête est partie et que la réponse du FDM est perdue, la commande de remboursement est annulée comme ci-dessus, mais la requête reste ouverte dans `pending_signs` et est renvoyée à l'identique avant toute autre signature (`DOCUMENT_PROCESSUS_TECHNIQUE_FR.md` §4). À la réponse tardive, `signing::apply` clôt la commande de remboursement, même annulée entre-temps, écrit son paiement et archive l'événement une seule fois. Une nouvelle demande de remboursement de la même vente retrouve ce résultat au lieu de rembourser deux fois.

Ces règles sont couvertes par `crates/pos-engine/tests/refund_guards.rs`.

M112A vs M112B (remboursement partiel combiné à une nouvelle vente)

Le formulaire de demande de certification exige de déclarer si le POS supporte **M112A** (un remboursement partiel combiné à des items de nouvelle vente sur un même ticket) ou **M112B** (remboursement seul, sur un ticket séparé).

Nous déclarons M112B : la commande de remboursement produit un **événement fiscal séparé** ne contenant que des lignes à quantités négatives, jamais mélangé à des lignes de nouvelle vente. C'est le pattern le plus simple et le plus sûr ; combiner remboursement et nouvelle vente sur un même ticket crée une particularité de piste d'audit que le manuel de certification demande explicitement aux candidats de reconnaître.

Les fixtures M112A et M112B se parsent toutes deux proprement (vérifié dans `uc_fixtures.rs`). Le mode déclaré ne contrôle que le flux UI.

---

2. Correction de paiement (hors périmètre v1.0.0)

La correction de paiement (M132, UC4118) n'est pas déclarée dans le périmètre de la v1.0.0. `Fdm-Client::sign_payment_correction` existe avec la forme `PaymentCorrectionInput`, mais aucune commande Tauri ni écran ne l'appelle : pour une erreur de moyen de paiement, l'opérateur utilise le remboursement puis une nouvelle vente.

---

3. Facture d'un remboursement (note de crédit)

Le POS n'émet pas de note de crédit autonome. Un remboursement est un événement **N** de montants négatifs (§1) ; il peut être repris sur une **facture** (événement I) avec d'autres ventes, ses montants comptant en négatif (cas UC4122 du jeu Advanced 1.6). Si la vente d'origine avait déjà été facturée, le remboursement figure sur une nouvelle facture : la première, en ajout seul, ne change pas. Le document Peppol est une facture dont la ligne remboursée garde une quantité négative. Voir *FACTURATION_PEPPOL_FR.md*.

---

4. Compteur sans trou

Voir `STRATEGIE_COMPTEURS_FR.md` §5. Le flux de remboursement passe par le même chemin que l'encaissement (numéro lu sous le verrou, compteur avancé dans la transaction qui archive) ; test de régression : `crates/pos-engine/tests/spf_e2e_no_gap.rs::refund_fdm_failure_does_not_advance_counter`.

---

5. Vérification

```
# Côté fixture : réponses remboursement + note de crédit parsent correctement
cargo test --package pos-fiscal --test uc_fixtures
cargo test --package pos-fiscal --test advanced_set_fixtures

# Côté e2e applicatif : remboursement complet, partiel et fdmRef
cargo test --package pos-engine --test spf_e2e_basic m111_
cargo test --package pos-engine --test spf_e2e_basic m112b_

# Côté no-gap : compteur intact si FDM échoue pendant remboursement
cargo test --package pos-engine --test spf_e2e_no_gap refund_fdm
```

Totaux : totaux par binaire de test dans *RAPPORT_TESTS_BASELINE_FR.md*, régénérés par `scripts/generate_test_rep`